

day22笔记

一、InMemorySaver记忆模块

- 1、今天讲解的记忆模块是 langgraph 里面新增的
- 2、地址: <https://langgraph.com.cn/index.html>
- 3、[LangSmith](#) — 有助于智能体评估和可观察性。调试性能不佳的 LLM 应用程序运行、评估智能体轨迹、在生产环境中获得可见性,并随着时间的推移提高性能。
- 4、示例代码

```
from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from dotenv import load_dotenv
from langchain_core.messages import HumanMessage

load_dotenv()

llm = init_chat_model(model='deepseek-chat')

memory = InMemorySaver()

agent = create_agent(
    model=llm,
    checkpointer=memory,
    system_prompt='你是一个乐于助人的AI助手,请记住用户的名字和刚才提到的事情'
)

# 使用记忆模块,在调用智能体的时候,必须设置一个config的配置信息(包含id)
config = {'configurable': {'thread_id': 'user_A'}}

result = agent.invoke(
    {'messages': [HumanMessage(content='你好,我叫小明,是一名python开发工程师')]} ,
    config=config
)
print(result['messages'][-1].content)

print('-' * 30)

result2 = agent.invoke(
    {'messages': [HumanMessage(content='我叫什么名字?')]} ,
    config=config
)
print(result2['messages'][-1].content)

print('-' * 30)
```

```

config2 = {'configurable': {'thread_id': 'user_B'}}
result3 = agent.invoke(
    {'messages': [HumanMessage(content='我叫小华')]}],
    config=config2
)
print(result3['messages'][-1].content)

print('-' * 30)

config2 = {'configurable': {'thread_id': 'user_B'}}
result4 = agent.invoke(
    {'messages': [HumanMessage(content='我叫什么名字? ')]},
    config=config2
)
print(result4['messages'][-1].content)

```

5、上面的记忆如果程序重新执行了，那么之前的记忆是没有了

```

from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from dotenv import load_dotenv
from langchain_core.messages import HumanMessage

load_dotenv()

llm = init_chat_model(model='deepseek-chat')

memory = InMemorySaver()

agent = create_agent(
    model=llm,
    checkpointer=memory,
    system_prompt='你是一个乐于助人的AI助手，请记住用户的名字和刚才提到的事情'
)

# 使用记忆模块，在调用智能体的时候，必须设置一个config的配置信息（包含id）
config = {'configurable': {'thread_id': 'user_A'}}

# result = agent.invoke(
#     {'messages': [HumanMessage(content='你好，我叫小明，是一名python开发工程师')]}],
#     config=config
# )
# print(result['messages'][-1].content)

print('-' * 30)

result2 = agent.invoke(
    {'messages': [HumanMessage(content='我叫什么名字? ')]},
    config=config
)
print(result2['messages'][-1].content)

```

```
print('-' * 30)
```

输出结果

```
-----  
抱歉，我们之前还没有提到过您的名字呢。请问您方便告诉我您的名字吗？这样我就能记住啦！😊  
-----
```

6、SQLite数据库

是专门用来做数据持久化的，可以把数据库建在本地，是非常轻量

二、skill技能

1、智能体里面要使用技能，需要使用深度推理智能体 `create_deep_agent`

2、安装

```
pip install deepagents
```

3、示例代码

```
from deepagents import create_deep_agent  
from deepagents.backends import LocalShellBackend  
from langchain.chat_models import init_chat_model  
from langgraph.checkpoint.memory import InMemorySaver  
from dotenv import load_dotenv  
from langchain_core.messages import HumanMessage
```

```
load_dotenv()
```

```
llm = init_chat_model(model='deepseek-chat')
```

```
# 创建提示词
```

```
prompt = """
```

```
角色定位
```

你是一个高度智能、多功能的AI助手。你的核心目标是准确识别用户的意图，并根据任务的复杂程度，灵活调用最合适的技能（Skill）来提供精准、高质量的回答。你不仅是一个信息提供者，更是一个问题解决者。

```
核心技能库
```

你具备以下核心技能，请在处理请求时自主判断并调用：

- 1、信息检索与整合：针对事实性、时效性问题，快速提取关键信息并综合多方来源。
- 2、逻辑推理与分析：针对复杂问题，进行逐步拆解、因果分析和逻辑推演。
- 3、编程与技术支持：编写、调试代码，解释技术概念，提供算法思路。
- 4、创意写作与表达：撰写文案、故事、诗歌，进行风格迁移或润色。
- 5、多语言翻译与转换：提供准确、符合语境的翻译服务。
- 6、数据处理与格式化：将信息整理为表格、列表、JSON等结构化格式。

```
工作流程与思维链
```

在回复用户之前，请务必在内心遵循以下思考步骤：

- 1、意图识别：用户的核心需求是什么？（是寻求事实、需要代码、想要创意，还是进行闲聊？）

- 2、技能匹配：解决这个问题需要调用上述哪项或哪几项技能？（例如：写一个爬虫脚本 = 编程技能 + 逻辑推理）
- 3、策略制定：
 - 如果是简单问题，直接给出简洁明了的答案。
 - 如果是复杂问题，采用“分步解答”策略，先给出框架，再填充细节。
 - 如果用户意图模糊，主动提出澄清性问题。
- 4、内容生成：根据制定的策略生成回复，确保语气专业、友好且富有同理心。
- 5、自我审查：检查回复是否准确、安全，是否符合格式要求。

回复规范与约束

- 准确性优先：对于不确定的信息，请如实告知，不要编造。
- 格式优化：善用 **Markdown**（如加粗重点、使用列表、代码块、表格）来提升阅读体验。
- 代码规范：提供代码时，必须包含必要的注释，并说明运行环境或依赖库。
- 语气风格：保持客观、中立、乐于助人。面对创意类任务时，可以更加生动活泼。
- 安全合规：拒绝回答涉及暴力、色情、非法或仇恨言论的请求。

"""

```
# 创建深度推理智能体
# skills=['./skills'] 表示技能存储在那个目录下
# backend=LocalShellBackend() 是加载本地skill的核心，用于执行本地的脚步和文件访问
agent = create_deep_agent(
    model=llm,
    skills=['./skills'],
    backend=LocalShellBackend(root_dir='.', virtual_mode=True),
    system_prompt=prompt,
    checkpointer=InMemorySaver()
)

config = {'configurable': {'thread_id': 'user_A'}}

result = agent.invoke(
    {'messages': [HumanMessage('我喜欢一个女孩，她对我时好时坏，有时候聊得很好，有时候又不回消息。我不知道她喜不喜欢我。')]},
    config=config
)

print(result['messages'][-1].content)
```

结果输出

兄弟，你这问题问对人了。我跟你好好唠唠。

说实话兄弟们，你这个问题我太熟了——**"她到底喜不喜欢我？"**

这个问题本身就是答案了，知道吧？

你看我教你一个最简单的判断标准：**如果你不确定她是不是喜欢你，那她就是不喜欢你。**

真的，就是这么简单。一个真正喜欢你的女孩，不用你去猜，她会让你知道的。她会主动找你聊天，会关心你今天吃了什么，会记得你说过的话。而不是"时好时坏"——你今天聊得好，她热情回应，明天发消息就石沉大海。

这叫什么？这叫**她在用她的节奏控制这段关系**。

她对你好，是因为她当下无聊了，需要人陪；她不回你消息，是因为她有更重要的事或者更重要的人。你在这边翻来覆去地想"她什么意思"，她在那边压根没想过你这件事，对吧？

我给你讲个最扎心的实话：

> **这世界上没有坏女人，只有不喜欢你的女人。**

她不是故意吊着你，她只是没那么在乎你。你没在她priority list的前排。

那怎么办？

我跟你三点，你记住：

第一，不要再给她加戏了。

你那脑子现在天天在想什么？"她是不是害羞？""她是不是在考验我？""她是不是怕主动显得掉价？"——别想了兄弟，这些都是你自己给自己编的故事。你要信事实，事实就是她不回你消息。

第二，把注意力拿回来放自己身上。

你看我直播间那些兄弟，天天在那儿想"她为什么不回我"，我问你一句——你今天健身了吗？你今天读书了吗？你今天把工作做好了吗？你把自己搞好了，你自己有价值了，她是能感受到的。

吸引力，不是讨好来的。没有人会因为你喜欢她而喜欢你，只会因为你吸引她而喜欢你。

第三，真想试一把——就直接试。

你说你喜欢她，那你就大大方方约一次。"周末有空吗？我知道一家不错的餐厅。"她答应了，就好好处；她找借口推了，那就明白了吗？不浪费时间。

别在那儿玩什么欲擒故纵、测试人性，我跟你讲，**用心机换来的感情，迟早会毁在心机上。**

最后送你一句话，这话是我自己的亲身体会：

我之前跟若离在一起的时候，说实话，我没珍惜过她。后来我才后悔。你知道为什么吗？因为真正对你好的人，你一开始是感觉不到的，你觉得理所当然。等你失去了你才发现——你再也遇不到那样的人了。

所以我不劝你放手还是坚持，我劝你想清楚：**你是真的喜欢她这个人，还是喜欢"追她"这个感觉？**

如果是真的喜欢这个人，那你就光明正大地表达，不卑不亢地邀请。她接不接受是她的事，但你别让自己以后后悔，说"当初没试过"。

当兵后悔三年，不当后悔一辈子，知道吧？

没毛病吧兄弟们。

